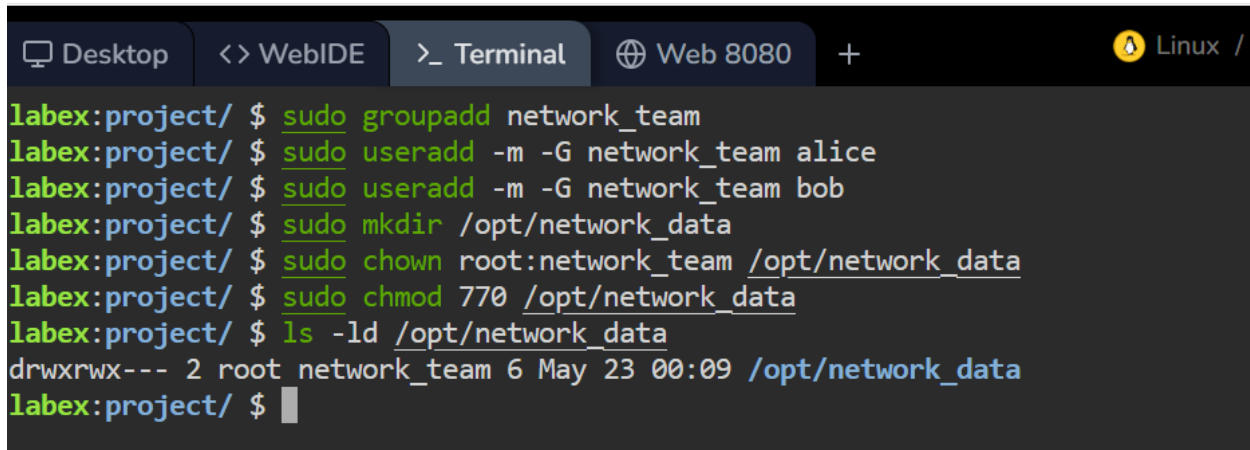


Assignment 15

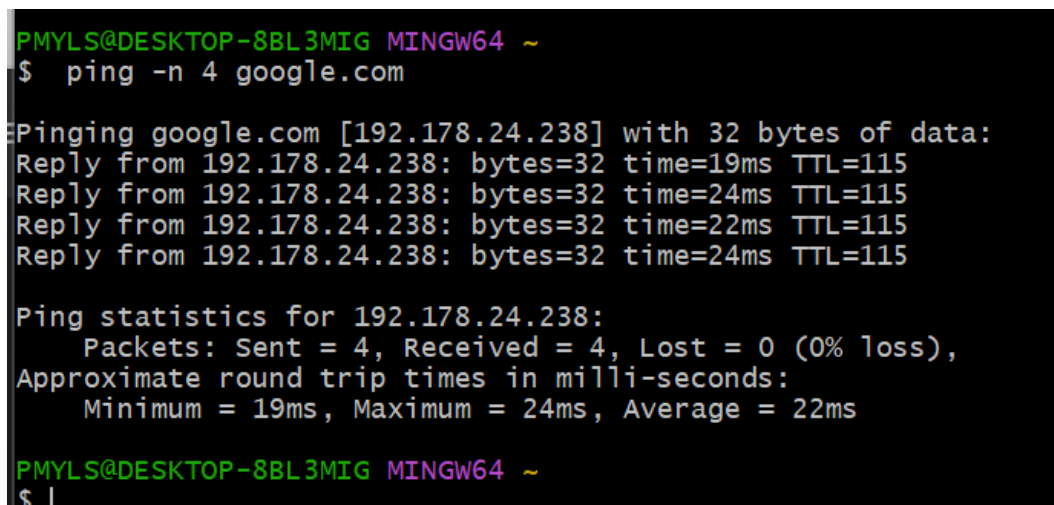
Part 1: User & Group Permissions

A terminal window with tabs for Desktop, WebIDE, Terminal, and Web 8080. The Terminal tab is active, showing a series of Linux commands and their output. The commands are: `sudo groupadd network_team`, `sudo useradd -m -G network_team alice`, `sudo useradd -m -G network_team bob`, `sudo mkdir /opt/network_data`, `sudo chown root:network_team /opt/network_data`, `sudo chmod 770 /opt/network_data`, and `ls -ld /opt/network_data`. The output of the last command shows the directory permissions as `drwxrwx--- 2 root network_team 6 May 23 00:09 /opt/network_data`.

```
labex:project/ $ sudo groupadd network_team
labex:project/ $ sudo useradd -m -G network_team alice
labex:project/ $ sudo useradd -m -G network_team bob
labex:project/ $ sudo mkdir /opt/network_data
labex:project/ $ sudo chown root:network_team /opt/network_data
labex:project/ $ sudo chmod 770 /opt/network_data
labex:project/ $ ls -ld /opt/network_data
drwxrwx--- 2 root network_team 6 May 23 00:09 /opt/network_data
labex:project/ $
```

A new group called `network_team` using `groupadd` is created, followed by the creation of two users, `alice` and `bob`, each added to the `network_team` group with the `useradd` command and the `-m` flag to create their home directories. A shared directory `/opt/network_data` is then created using `mkdir`. Ownership of this directory is changed so that the group owner is `network_team` using `chown`, and the permissions are set with `chmod 770` to allow full access to the owner and group members, while denying access to others. Finally, `ls -ld` is used to verify and display the directory's permissions and ownership, confirming that the setup has been applied correctly.

Part 2: Network Tools & Real-Time Checks

A terminal window showing the output of a `ping -n 4 google.com` command. The output shows four successful replies from IP address 192.178.24.238 with varying response times and TTL values. Below the replies, the ping statistics are displayed, showing 4 packets sent, 4 received, and 0 lost (0% loss), with an average round trip time of 22ms.

```
PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ ping -n 4 google.com

Pinging google.com [192.178.24.238] with 32 bytes of data:
Reply from 192.178.24.238: bytes=32 time=19ms TTL=115
Reply from 192.178.24.238: bytes=32 time=24ms TTL=115
Reply from 192.178.24.238: bytes=32 time=22ms TTL=115
Reply from 192.178.24.238: bytes=32 time=24ms TTL=115

Ping statistics for 192.178.24.238:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 19ms, Maximum = 24ms, Average = 22ms

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ |
```

The `ping -c 4 google.com` command sends four ICMP echo request packets to google.com to test network connectivity and measure the round-trip time to the host. It helps determine if a remote host is reachable and assess network latency.

```
labex:project/ $ sudo netstat -tuln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp    0      0 127.0.0.11:33777        0.0.0.0:*               LISTEN
tcp    0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp    0      0 0.0.0.0:3001            0.0.0.0:*               LISTEN
tcp    0      0 0.0.0.0:3002            0.0.0.0:*               LISTEN
tcp6   0      0 :::22                   :::*                     LISTEN
udp    0      0 127.0.0.11:46666        0.0.0.0:*               *
udp    0      0 0.0.0.0:3001            0.0.0.0:*               *

labex:project/ $ sudo ss -tulwn
Netid      State      Recv-Q     Send-Q     Local Address:Port      Peer Address:Port      Process
udp        UNCONN     0           0           127.0.0.11:46666        0.0.0.0:*
udp        UNCONN     0           0           0.0.0.0:3001            0.0.0.0:*
tcp        LISTEN     0           4096        127.0.0.11:33777        0.0.0.0:*
tcp        LISTEN     0           128         0.0.0.0:22              0.0.0.0:*
tcp        LISTEN     0           5           0.0.0.0:3001            0.0.0.0:*
tcp        LISTEN     0           128         0.0.0.0:3002            0.0.0.0:*
tcp        LISTEN     0           128         [::]:22                  [::]:*
```

These commands, `netstat -tuln` and `ss -tulwn`, display a list of all active network connections and listening ports on the system. They show which services are running and on what IP addresses and ports they are awaiting incoming connections for both TCP and UDP protocols.

```
MINGW64:/c/Users/PMLS

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ ipconfig

Windows IP Configuration

Ethernet adapter VirtualBox Host-Only Network:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::7d5e:a5bf:c9b7:aac2%1
    IPv4 Address. . . . . : 192.168.56.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 

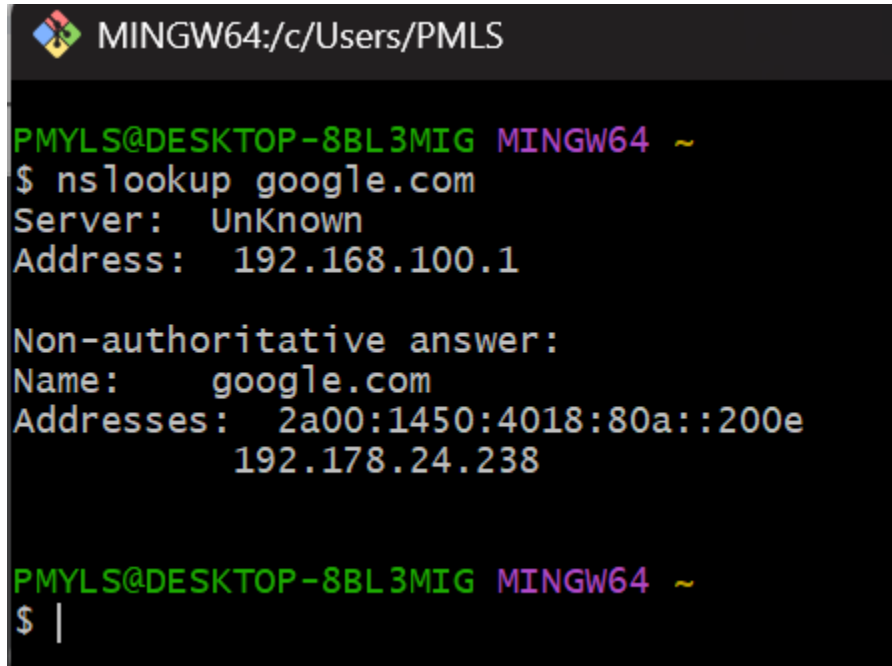
Wireless LAN adapter Local Area Connection* 9:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 

Wireless LAN adapter Local Area Connection* 10:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :
```

The ipconfig command displays the current TCP/IP network configuration for all network adapters on a Windows system. It shows details like IP addresses, subnet masks, and default gateways, which are crucial for network troubleshooting and understanding connectivity.



```
MINGW64:/c/Users/PMLS
PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ nslookup google.com
Server:    UnKnown
Address:   192.168.100.1

Non-authoritative answer:
Name:      google.com
Addresses: 2a00:1450:4018:80a::200e
           192.178.24.238

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ |
```

The nslookup google.com command queries the Domain Name System (DNS) to resolve the domain name google.com into its corresponding IP addresses. It also identifies the DNS server that provided the resolution.

Part 3: Compression & Decompression

This sequence of commands creates a directory, populates it with files, then archives and compresses the directory, followed by decompression and extraction:

1. `mkdir -p ./opt/network_data & echo "... " > ./opt/network_data/logX.txt`: Creates the network_data directory and populates it with three sample log files.
2. `ls ./opt/network_data & cat ./opt/network_data/log2.txt`: Verifies the creation and content of the files within the directory.
3. `tar cvf network_data.tar ./opt/network_data`: Archives the network_data directory and its contents into a single file named network_data.tar.
4. `gzip network_data.tar`: Compresses the network_data.tar archive into network_data.tar.gz.

```

MINGW64:/c/Users/PMLS

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ mkdir -p ./opt/network_data

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ echo "Log file 1: Network up" > ./opt/network_data/log1.txt

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ echo "Log file 2: Error detected" > ./opt/network_data/log2.txt

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ echo "Log file 3: Bandwidth usage normal" > ./opt/network_data/log3.txt

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ ls ./opt/network_data
log1.txt  log2.txt  log3.txt

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ cat ./opt/network_data/log2.txt
Log file 2: Error detected

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ tar cvf network_data.tar ./opt/network_data
./opt/network_data/
./opt/network_data/log1.txt
./opt/network_data/log2.txt
./opt/network_data/log3.txt

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ gzip network_data.tar

```

Then:

1. `ls -lh`: Shows the file sizes, confirming the `network_data.tar.gz` compressed file.
2. `gunzip network_data.tar.gz`: Decompresses the archive, restoring `network_data.tar`.
3. `tar xvf network_data.tar`: Extracts the contents from the `network_data.tar` archive, recreating the original `network_data` directory and its files.

```

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ gunzip network_data.tar.gz

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ tar xvf network_data.tar
./opt/network_data/
./opt/network_data/log1.txt
./opt/network_data/log2.txt
./opt/network_data/log3.txt

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$

```

Then:

This series of commands demonstrates archiving a directory and then compressing/decompressing it using bzip2.

1. `tar cvf network_data.tar ./opt/network_data`: This command creates a tar archive named `network_data.tar` from the `./opt/network_data` directory, listing its contents during the process.
2. `bzip2 network_data.tar`: This command compresses the `network_data.tar` archive using the bzip2 algorithm, replacing the original file with `network_data.tar.bz2`.
3. `bunzip2 network_data.tar.bz2`: This command decompresses the `network_data.tar.bz2` file, restoring the `network_data.tar` archive.
4. `tar xvf network_data.tar`: This command extracts the contents of the `network_data.tar` archive, recreating the `./opt/network_data` directory and its files.

```
PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ tar cvf network_data.tar ./opt/network_data
./opt/network_data/
./opt/network_data/log1.txt
./opt/network_data/log2.txt
./opt/network_data/log3.txt

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ bzip2 network_data.tar

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ bunzip2 network_data.tar.bz2

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$ tar xvf network_data.tar
./opt/network_data/
./opt/network_data/log1.txt
./opt/network_data/log2.txt
./opt/network_data/log3.txt

PMYLS@DESKTOP-8BL3MIG MINGW64 ~
$
```

Part 4: Text Processing with grep & awk

This series of commands demonstrates text processing operations using grep and awk on a custom log file.

1. `mkdir logs & cat > logs/syslog <<EOF ... EOF`: Creates a directory named logs and then creates a multi-line syslog file within it. The `cat <<EOF ... EOF` syntax is a "here document" used to easily input multiple lines of text into a file.
2. `grep "error" logs/syslog`: Searches for and displays all lines containing the string "error" in the logs/syslog file.
3. `grep -c "error" logs/syslog`: Counts and displays the total number of lines that contain the string "error" in the logs/syslog file.
4. `grep "error" logs/syslog | awk '{print $1, $2, $3, $5}'`: Filters lines containing "error" from syslog and then uses awk to extract and print the 1st (Month), 2nd (Day), 3rd (Time), and 5th (Process/Source) fields of each matching line.
5. `grep "error" logs/syslog | awk '{print $5}' | sort | uniq -c | sort -nr`: This command pipeline first extracts lines with "error," then uses awk to isolate the 5th field (process/source). These sources are then sorted, unique occurrences are counted (`uniq -c`), and finally, the results are sorted numerically in reverse order (`sort -nr`) to show the most frequent error sources.

MINGW64:/c/Users/PMLS

PMYLS@DESKTOP-8BL3MIG MINGW64 ~

```
$ grep "error" /var/log/syslog
grep: /var/log/syslog: No such file or directory
```

PMYLS@DESKTOP-8BL3MIG MINGW64 ~

```
$ mkdir logs
cat > logs/syslog <<EOF
May 22 10:05:23 localhost kernel: Booting Linux
May 22 10:05:24 localhost systemd: Starting service...
May 22 10:06:01 localhost kernel: error: Disk read failure
May 22 10:06:02 localhost NetworkManager: error: Network unreachable
May 22 10:06:03 localhost systemd: Service started
May 22 10:06:04 localhost kernel: error: Buffer overflow
EOF
```

PMYLS@DESKTOP-8BL3MIG MINGW64 ~

```
$ grep "error" logs/syslog
May 22 10:06:01 localhost kernel: error: Disk read failure
May 22 10:06:02 localhost NetworkManager: error: Network unreachable
May 22 10:06:04 localhost kernel: error: Buffer overflow
```

PMYLS@DESKTOP-8BL3MIG MINGW64 ~

```
$ grep -c "error" logs/syslog
3
```

PMYLS@DESKTOP-8BL3MIG MINGW64 ~

```
$ grep "error" logs/syslog | awk '{print $1, $2, $3, $5}'
May 22 10:06:01 kernel:
May 22 10:06:02 NetworkManager:
May 22 10:06:04 kernel:
```

PMYLS@DESKTOP-8BL3MIG MINGW64 ~

```
$ grep "error" logs/syslog | awk '{print $5}' | sort | uniq -c | sort -nr
      2 kernel:
      1 NetworkManager:
```

PMYLS@DESKTOP-8BL3MIG MINGW64 ~

```
$ |
```